KENNESAW STATE
UNIVERSITY

CS 4277
Regularization

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

College of Computing and Software Engineering

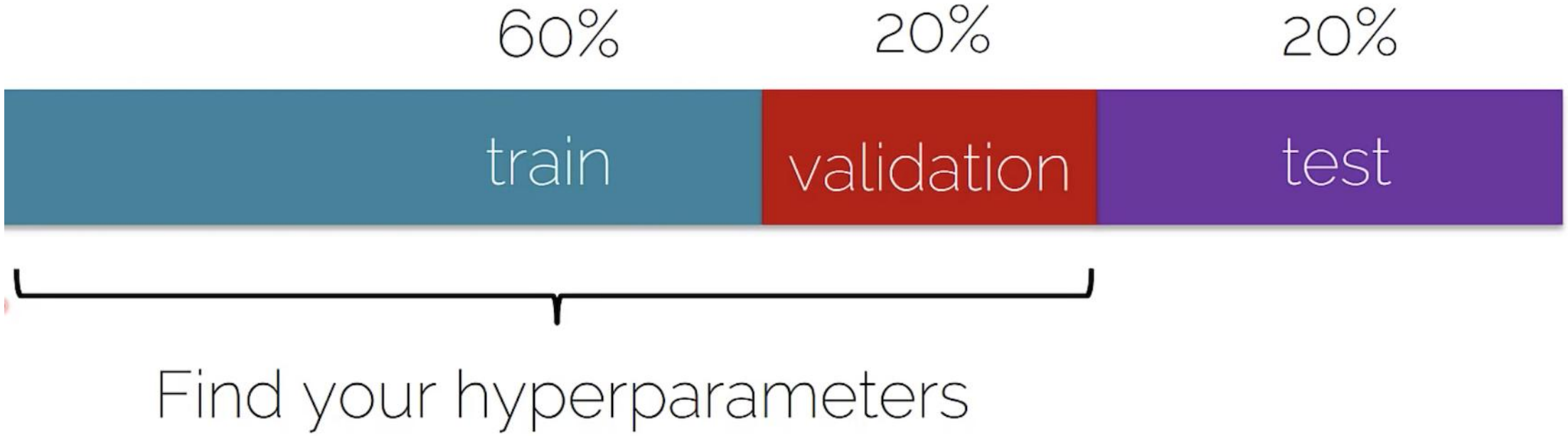
Learning Objectives

- Basic ML Concepts
- Overfitting and Underfitting
- Model Capacity
- Prevent Overfitting -- Regularization
  - Weight regularization structures
  - Dropout
  - Weight-decay
  - Batch normalization
  - Early stopping

Feedforward networks

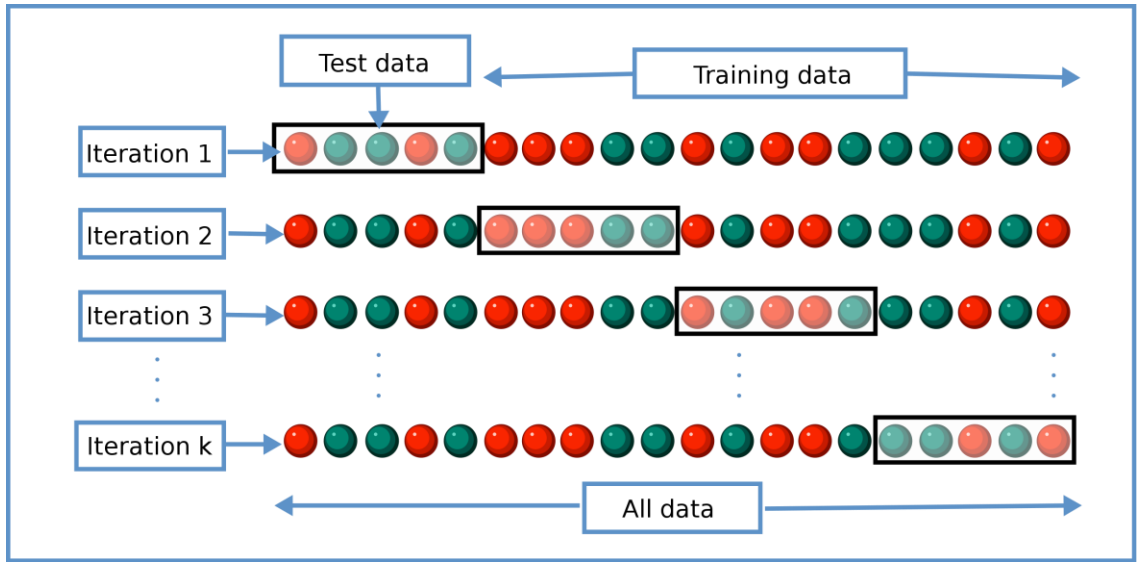
Basic Recipe for ML

Split your data



Basic Recipe for ML: K-fold Cross Validation

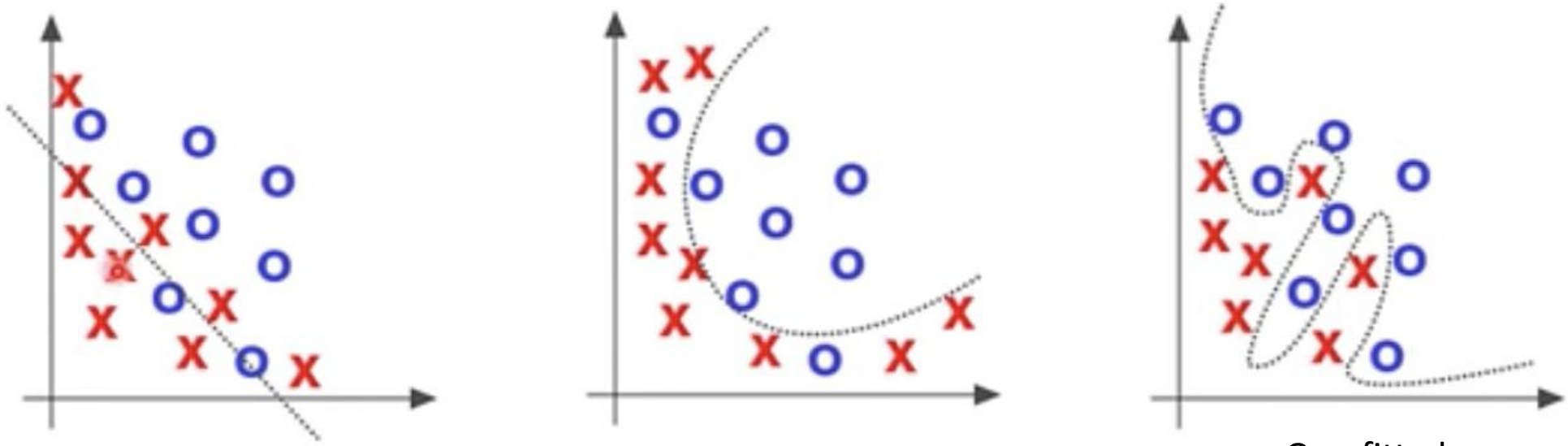
- Useful when not sufficient data
- Algorithm:
  - Partition the training data into K parts
  - For i = 1, …, K
    - Use the i-th part as the validation set, the rest for training
  - Report the averaged the K validation errors
- Popular choices: K = 5 or 10



Training Error and Generalization Error

- Training error: model error on the training data
- Generalization error: model error on new data
- Example: practice a future exam with past exams
  - Doing well on past exams (training error) doesn't guarantee a good score on the future exam (generalization error)
  - Student A gets 0 error on past exams by rote learning
  - Student B understands the reasons for given answers

Overfitting and Underfitting



Underfitted

Appropriate

Overfitted

Src: Deep learning by adam gibson, josh patterson O'reily media inc. 2017

Underfitting and Overfitting

- Underfitting: The classifier learned on the training set is not expressive enough to even account for the data provided. In this case, both the training error and the test error will be high, as the classifier does not account for relevant information present in the training set.
- Overfitting: The classifier learned on the training set is too specific and cannot be used to accurately infer anything about unseen data. Although training error continues to decrease over time, test error will begin to increase again as the classifier begins to make decisions based on patterns which exist only in the training set and not in the broader distribution.

Underfitting and Overfitting

Data complexity

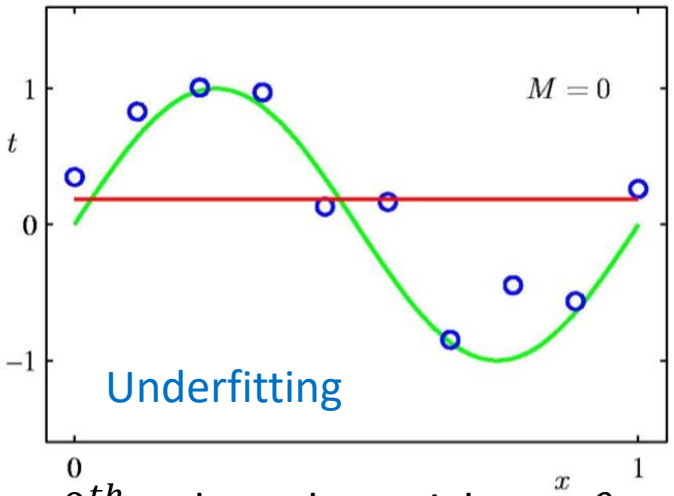
		Simple	Complex
Model capacity	Low	Normal	Underfitting
	High	Overfitting	Normal

Model Capacity

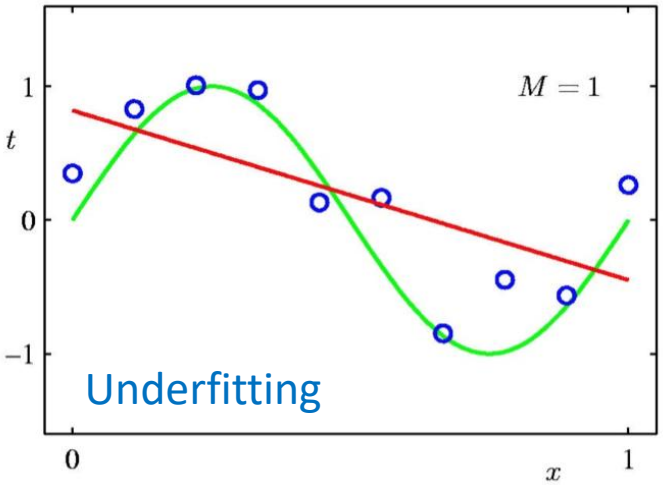
- The ability to fit variety of functions
- Low-capacity models struggles to fit training set
  - Underfitting
- High-capacity models can memorize the training set
  - Overfitting

Example of model capacity vs. performance

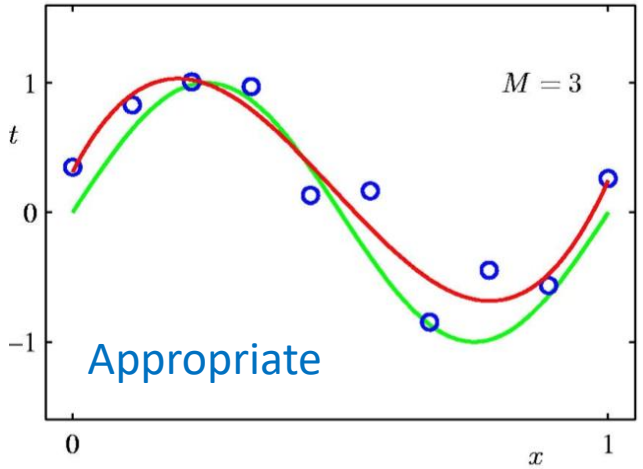
- Linear regression: $y = \theta^T x$
- Polynomial Regression:
  - $y = \theta^T \phi(x)$
  - $\phi(x) = [1, x_1, x_2, x_1^2, x_2^2]$
  - $y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2$



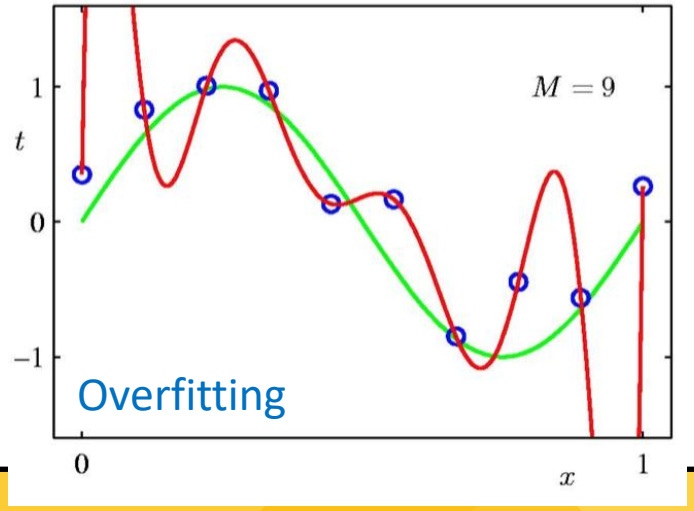
0th order polynomial, $y = \theta_0$



1st order polynomial, $y = \theta_0 + \theta_1 x_1$



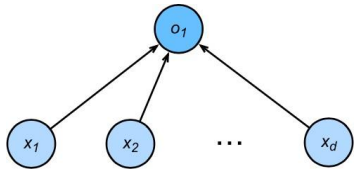
3rd order polynomial, $y = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^3$

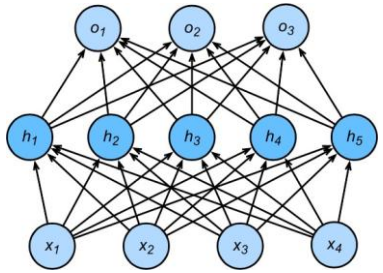


9th order polynomial

Estimate Model Capacity

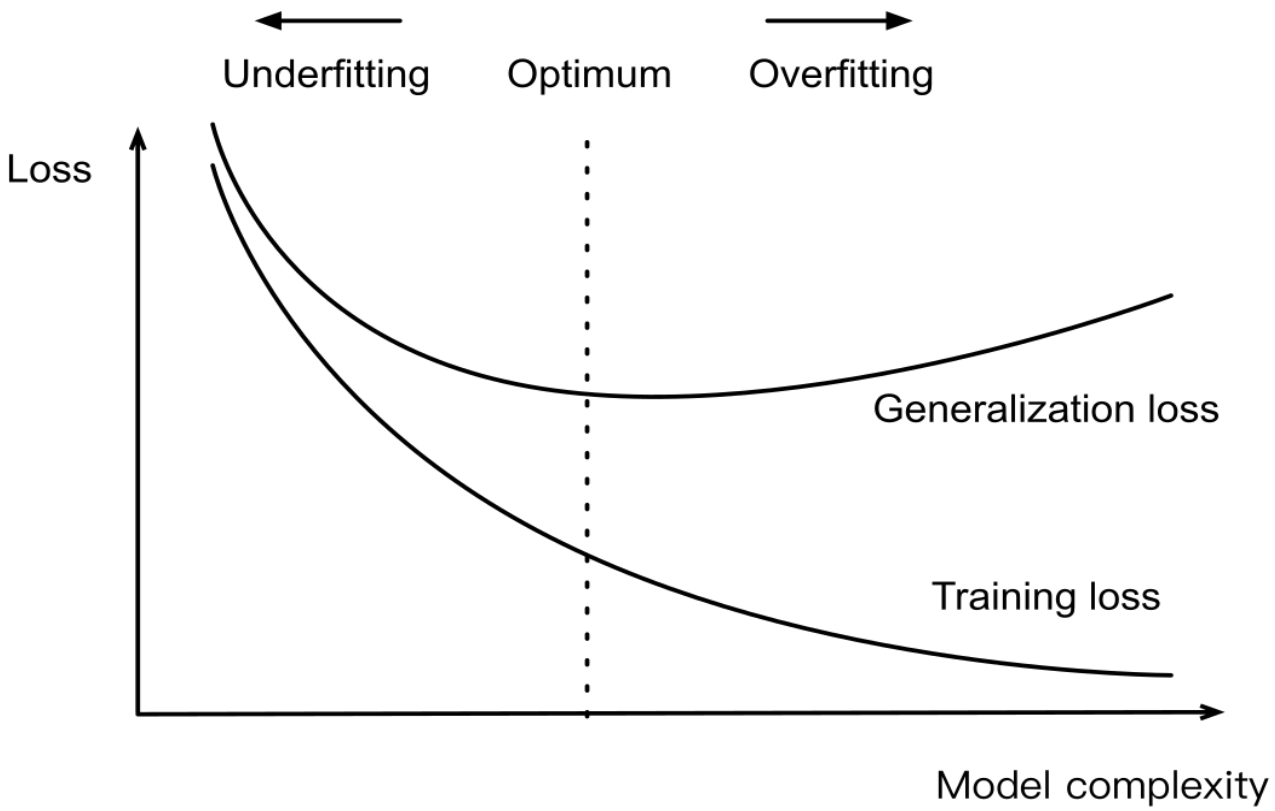
- It's hard to compare complexity between different algorithms
  - e.g., tree vs neural network





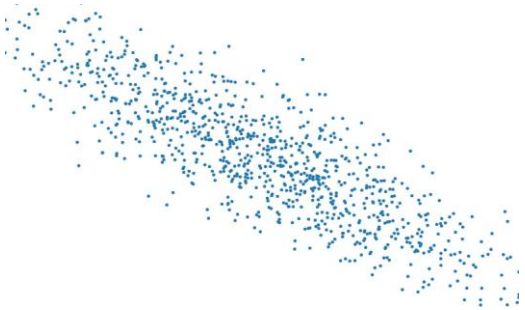
- Given an algorithm family, two main factors matter:
  - The number of parameters
  - The values taken by each parameter

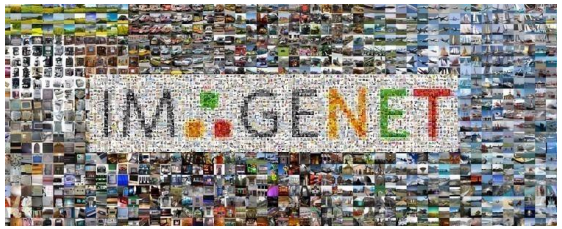
Influence of Model Complexity



Data Complexity

- Multiple factors matters
  - # of examples
  - # of elements in each example
  - time/space structure
  - diversity





Prevent Overfitting

- Approach 1: Get more data!
  - Always best if possible!
  - If no natural ones, use data augmentation
- Approach 2: Average many different models.
  - Models with different forms.
  - Train on different subsets
- Approach 3: reduce number of features
- Approach 4: Use a model that has the right capacity:
  - enough to fit the true regularities.
  - not enough to also fit spurious regularities (if they are weaker).
  - Parameter tuning
- Approach 5: Use specific regularizing structures

Regularization: Preventing Overfitting

- Regularization
  - Keep all features, but reduce magnitude of parameters θ
  - Works well when we have a lot of features, each of which contributes a bit to predicting y
- Specifically: additional terms in the training optimization objective to prevent overfitting or help the optimization
- Methods:
  - Data augmentation
  - Weight-decay
  - Weight regularization structures
  - Early stopping
  - Dropout
  - Batch normalization

Linear regression with Regularization

Linear Regression with Regularization (Ridge Regression)

- A model with smaller parameters is more biased but less sensitive to fluctuations in the training data, reducing the risk of overfitting.
- Motivation: Penalize large parameters
- How: Ridge Regression – L2 Regularizer

$$L(\theta) = \sum_i^n \left|\left|y_i - \theta^T \phi(x_i)\right|\right|^2 + \frac{\lambda}{2}\theta^T\theta, \text{ s.t. } \lambda > 0$$

- Regularization term $\frac{\lambda}{2}\theta^T\theta \Longrightarrow \left|\left|\theta\right|\right|_2^2$

Linear Regression with Regularization (Lasso Regression)

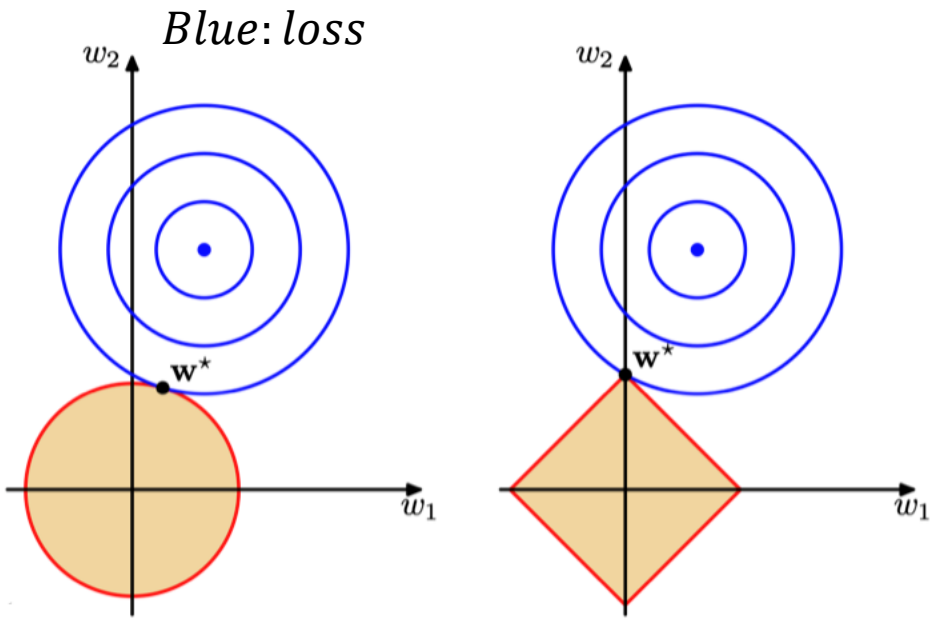
- Motivation: Penalize large parameters
- How: L1 Regularizer

$$L(\theta) = \sum_{i}^{n} \left|\left|y_i - \theta^T \phi(x_i)\right|\right|^2 + \frac{\lambda}{2} \|\theta\|_1$$

- LASSO: least absolute shrinkage and selection operator

Ridge Regression vs Lasso

- Ridge Regression
  - Prefer parameters close to zero
- Lasso
  - Yield sparse parameters (exact zero)
  - Used in feature selection

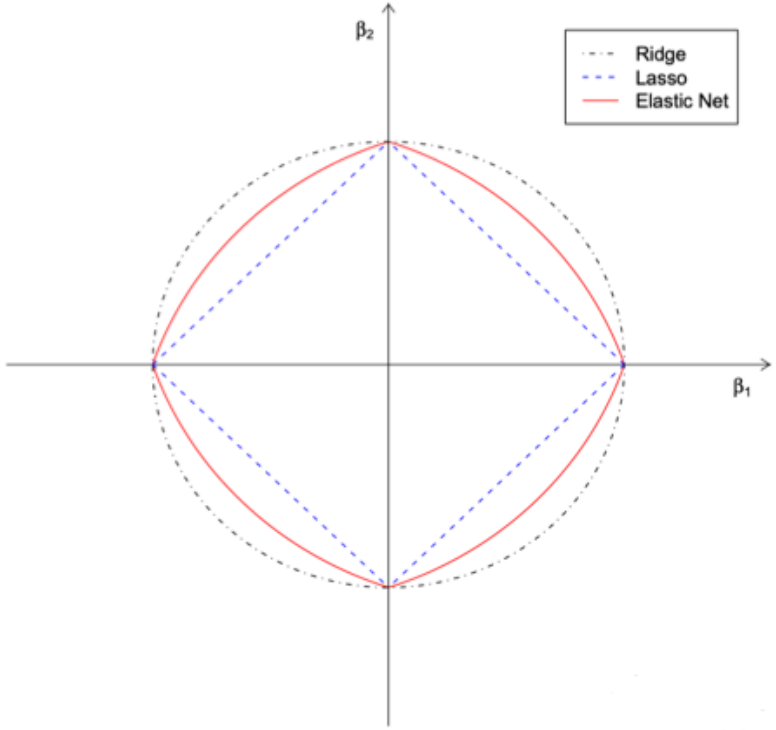


Ridge Regression Lasso Regression

w^* is the optimal solution

Elastic net (L1+L2)

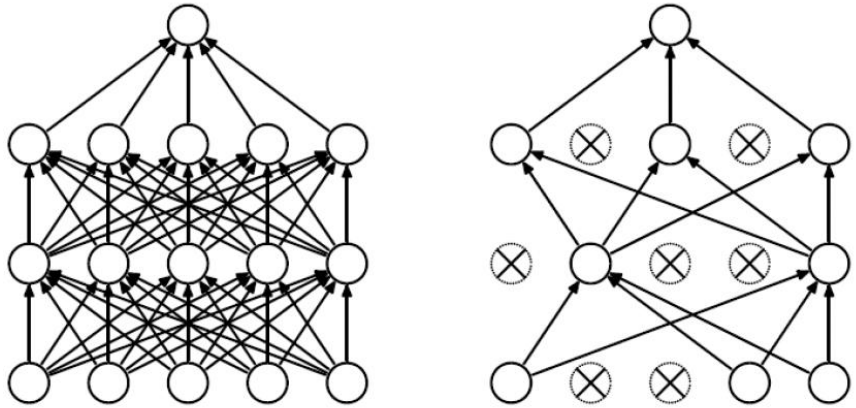
- Elastic net: combine the L1 regularization with the L2 regularization
- $L(\theta) = \sum_i^n \left|\left|y_i - \theta^T \phi(x_i)\right|\right|^2 + \frac{\lambda_1}{2}\|\theta\|_1 + \frac{\lambda_2}{2}\|\theta\|_2^2$



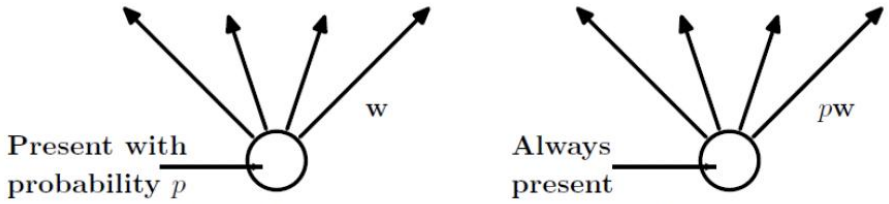
Dropout

Dropout: An efficient way to average many large neural nets

- Consider a neural net with one hidden layer.
- Each time we present a training example, we randomly omit each hidden unit with probability 0.5.
- So we are randomly sampling from 2^H different architectures. All architectures share weights.
- While training, dropout is implemented by only keeping a neuron active with some probability p (a hyperparameter), or setting it to zero otherwise.



(a) Standard Neural Net (b) After applying dropout.



(a) At training time (b) At test time

Dropout implementation

- Note that in the predict function we are not dropping anymore, but we are performing a scaling of both hidden layer outputs by p.
- At test time all neurons see all their inputs
  - the outputs of neurons at test time to be identical to their expected outputs at training time
- For example, in case of p=0.5, the neurons must halve their outputs at test time to have the same output as they had during training time (in expectation)
  - Consider an output of a neuron x (before dropout)
  - With dropout, the expected (Avg.) output from this neuron will become $px+(1-p)0$, because the neuron's output will be set to zero with probability $1-p$
  - At test time, when we keep the neuron always active, we must adjust $x \rightarrow px$ to keep the same expected output.

```python
""" Vanilla Dropout: Not recommended implementation (see notes below) """

p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  """ X contains the data """

  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = np.random.rand(*H1.shape) < p # first dropout mask
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = np.random.rand(*H2.shape) < p # second dropout mask
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)

def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
  H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
  out = np.dot(W3, H2) + b3
```

Dropout implementation

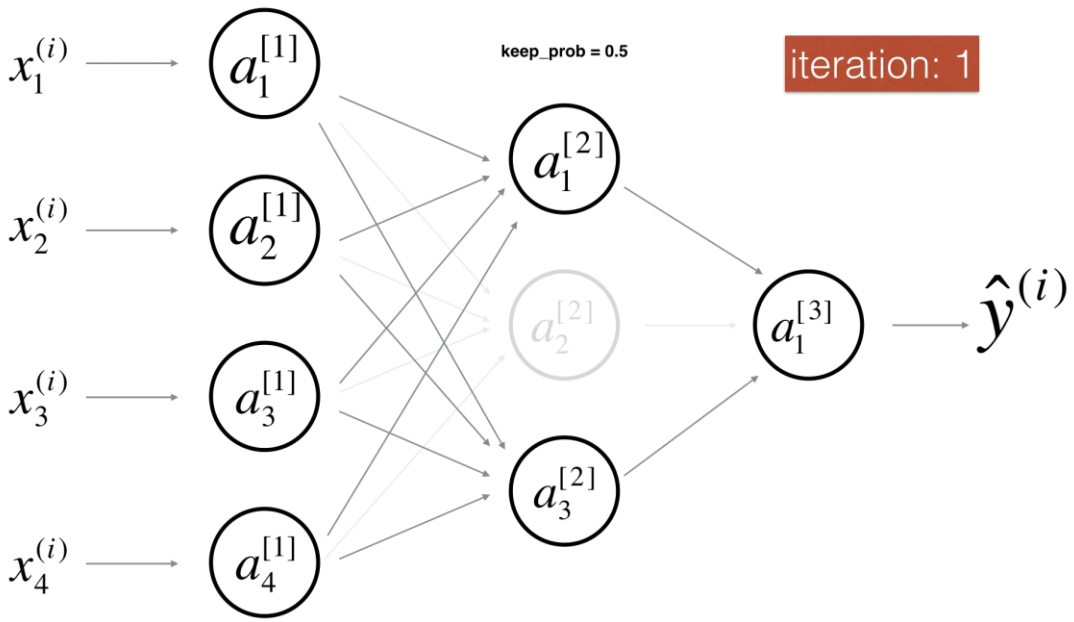
- Inverted Dropout:
  - which performs the scaling at train time
  - leaving the forward pass at test time untouched

```
"""
Inverted Dropout: Recommended implementation example.
We drop and scale at train time and don't do anything at test time.
"""

p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = (np.random.rand(*H1.shape) < p) / p # first dropout mask. Notice /p!
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = (np.random.rand(*H2.shape) < p) / p # second dropout mask. Notice /p!
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)

def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) # no scaling necessary
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  out = np.dot(W3, H2) + b3
```

Dropout as a form of model averaging

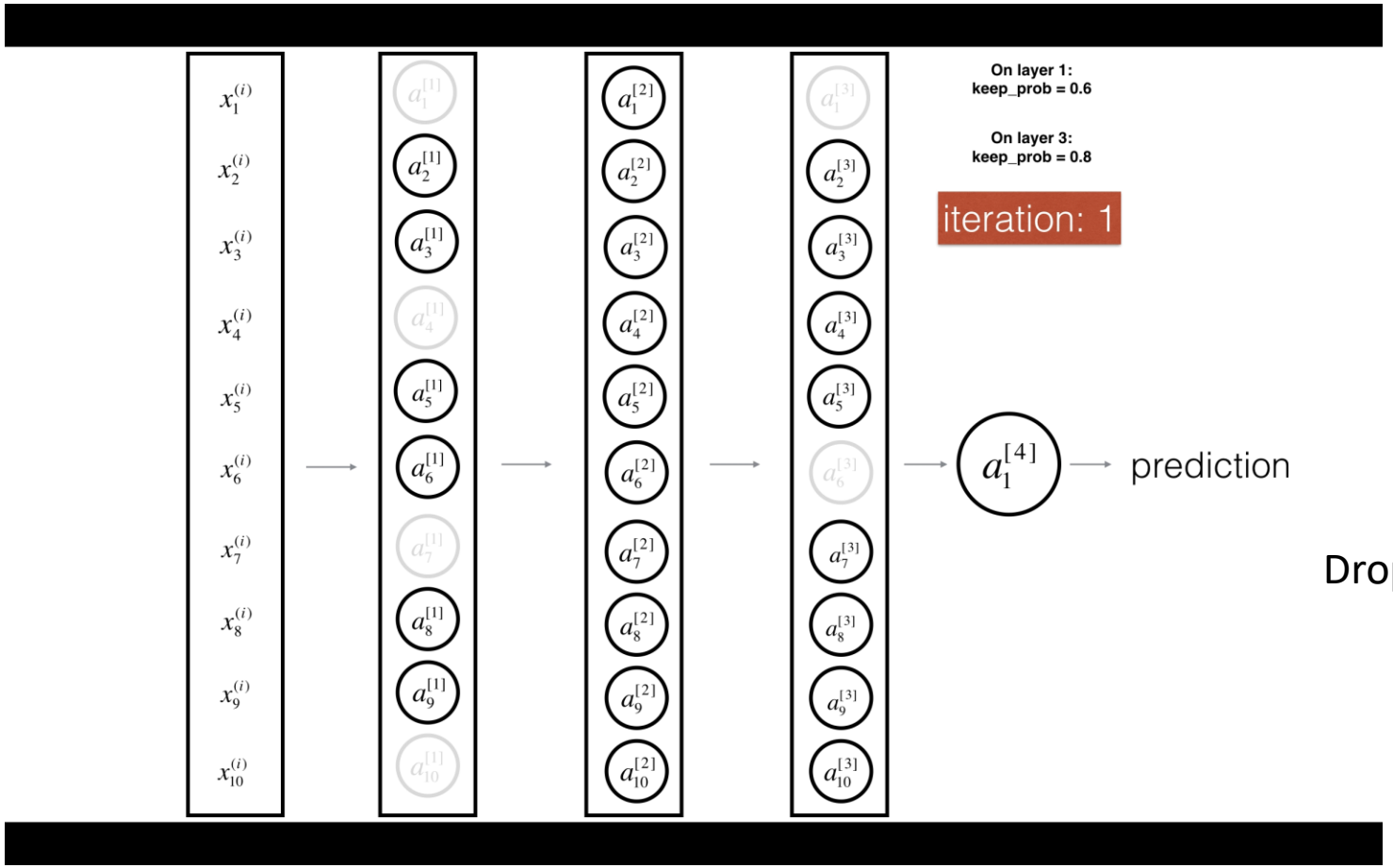
- We sample from 2^H models. So only a few of the models ever get trained, and they only get one training example.
- The sharing of the weights means that every model is very strongly regularized.
  - It's a much better regularizer than L2 or L1 penalties that pull the weights towards zero.

Dropout Visualization



Drop-out on the second hidden layer

Dropout Visualization



Drop-out on the first and third hidden layers.

Some dropout tips

- Dropout lowers your capacity
  - Increase network size by n/p where n is # hidden units in original, p is probability of dropout
- Dropout adds noise to gradients
  - Increase learning rate by 10-100
  - Or increase momentum (e.g. from 0.9 to 0.99)
  - These can cause large weight growths, use weight regularization

Weight decay

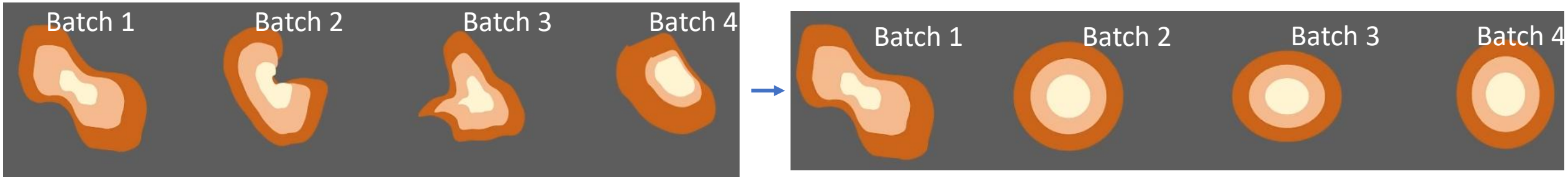
Weight decay

- Another approach to prevent over-fitting is weight-decay: all weights are multiplied by some fraction in (0,1) after every epoch.
  - Encourages smaller weights and less complex hypothesis
  - Equivalently: change Error function to include a term for the sum of the squares of the weights in the network. (general)

Batch Normalization

Batch Normalization

- Motivation: Addressing internal covariate shift among different batch of training samples



- Batch normalization, it is a process to make neural networks faster and more stable through adding extra layers in a deep neural network.
- The new layer performs the standardizing and normalizing operations on the input of a layer coming from a previous layer.

Batch normalization: Whitening

- It makes sense to normalize the output of each layer
  - 0 Mean, 1 standard deviation
  - Empirically observed as improving convergence
- Latter layers can be considered “using previous layer’s output to perform machine learning”
- How to do this during stochastic mini-batch optimization?
  - “Approximate mean and standard deviation” using mini-batch

Batch Normalization: Imperfect approximation

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

- Use mini-batch to approximate E and Var
- What if it's wrong?
  - Make sure the network can "correct" this change

$$y^{(k)} = \gamma^{(k)}\widehat{x}^{(k)} + \beta^{(k)}.$$

  - 2 learnable parameters for each x

Batch Normalization

- This is done for each hidden dimension separately

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

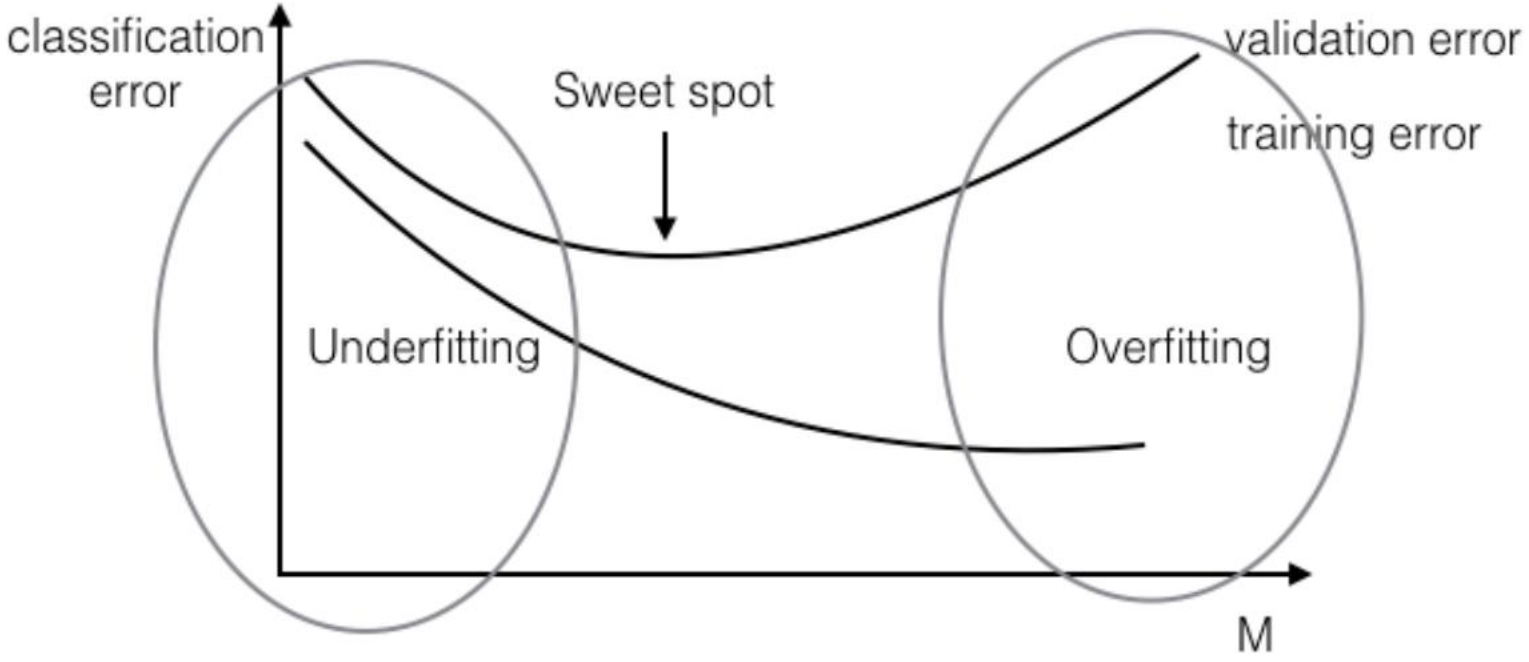
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Early Stopping

- Stop your optimization after M (>= 0) number of gradient steps, even if optimization has not converged yet.



Implementation

- D2L

- D2L
- Custom Layers:
- D2L
- Loading and Saving Tensors:
- D2L
- Fisseha Berhane
- D2L

Courtesy

- Andrej Karpathy, Zsolt Kira, Fuxin Li, Andrew ng
- Hafiz Khan